

SQL DML, 트랜잭션, DDL 핵심정리

오늘의 강의 학습 요약

금일 강의는 SQL의 **DML(INSERT/UPDATE/DELETE)** 을 중심으로, DML이 반드시 함께 이해되어야 하는 **트랜잭션(Transaction)** 개념을 사례(계좌이체, 여행 예약, 회원 탈퇴)로 설명한 뒤, MySQL 환경에서의 **오토커밋(auto-commit)** 특성과 명시적 커밋/롤백 처리 방법을 정리하는 흐름으로 진행되었습니다.

이후 INSERT 문법(단일행/다중행)과 UPDATE/DELETE의 조건 유무에 따른 동작 차이를 실습으로 확인하고, DML 결과가 커밋 전에는 세션(접속) 범위에서만 보이며 다른 세션에는 보이지 않는다는 점을 CLI 접속으로 검증했습니다. 마지막으로 DDL의 의미와 **DDL은 실행 즉시 자동 커밋되어 롤백이 불가**하다는 핵심 특징을 강조했습니다.

1. DML과 트랜잭션(TCL)

DML(INSERT/UPDATE/DELETE)은 트랜잭션으로 묶여 논리적 1작업으로 처리되며, 모두 성공 시 COMMIT으로 반영되고 일부라도 실패 시 ROLLBACK으로 취소되어 데이터 일관성을 보장합니다.

DML은 데이터를 “조회”하는 SELECT와 달리, 데이터를 “변경”하는 작업입니다. 따라서 변경 작업이 중간 실패로 불완전하게 남으면 데이터 무결성이 깨지기 쉽습니다. 이 문제를 해결하기 위해 DML은 트랜잭션과 결합되어 이해되어야 합니다.

DML의 범주는 다음 세 가지로 정리됩니다.

- INSERT는 새로운 레코드를 생성하는 문장입니다.
- UPDATE는 기존 레코드를 수정하는 문장입니다.
- DELETE는 기존 레코드를 삭제하는 문장입니다.

트랜잭션은 **한 개 이상의 DML을 논리적으로 하나의 작업 단위로 묶는 개념**입니다. 물리적으로는 여러 문장이지만, 비즈니스 관점에서는 “하나의 일”로 취급되어야 하는 상황에서 사용됩니다.

트랜잭션의 필요성은 “부분 성공”이 허용되지 않는 사례에서 명확해집니다. 예를 들어 계좌이체는 걸보기엔 1개의 작업이지만, 내부적으로는 출금 계좌 UPDATE와 입금 계좌 INSERT/UPDATE가 함께 성공해야만 합니다. 하나만 성공하면 금액 불일치가 발생하므로 반드시 한 묶음으로 성공/실패가 결정되어야 합니다.

강의에서 사용한 대표 예시는 다음과 같이 요약됩니다.

대표 상황	내부 DML 관점	트랜잭션이 필요한 이유
계좌이체(A→B 500)	A는 UPDATE가 필요하고 B는 INSERT/UPDATE가 필요합니다.	한쪽만 성공하면 잔액 불일치가 발생합니다.
제주도 여행 준비	예약 단계별로 여러 변경이 발생할 수 있습니다.	비행기 실패 시 숙소/렌트카/식당 예약을 취소해야 합니다.
회원 탈퇴	회원 DELETE와 게시글 DELETE 등이 연쇄됩니다.	회원만 삭제되고 게시글 삭제가 실패하면 데이터 정합성이 깨집니다.

트랜잭션 제어는 SQL에서 **TCL**(Transaction Control Language)로 수행하며, 핵심 명령은 COMMIT과 ROLLBACK입니다.

- COMMIT은 트랜잭션의 변경 사항을 DB에 확정 반영합니다.
- ROLLBACK은 트랜잭션 시작 이후의 변경 사항을 취소하고 이전 상태로 되돌립니다.

트랜잭션에는 “시작과 종료”라는 범위(바운더리) 개념이 필요합니다. 강의에서는 다음 규칙을 강조했습니다.

- 트랜잭션은 DML을 실행하는 순간 자동으로 시작됩니다.
- 트랜잭션은 COMMIT 또는 ROLLBACK으로 종료됩니다.
- 트랜잭션은 여러 DML뿐 아니라 DML 1개만으로도 성립할 수 있습니다.

특히 여러 DML을 실행한 뒤 COMMIT/ROLLBACK을 언제 하느냐에 따라 반영 범위가 달라집니다. 예를 들어 INSERT 후 COMMIT을 한 번 수행하면, 이후 UPDATE를 실행하고 ROLLBACK하더라도 UPDATE만 취소되고 INSERT는 유지됩니다.

또한 강의에서는 **DDL(CREATE 등)**은 트랜잭션과 무관하게 자동 커밋되는 특성이 있어, DML을 롤백하려던 중간에 DDL을 실행하면 의도치 않게 DML까지 커밋되는 실수 가능성을 경고했습니다.

핵심 키워드

#DML

#트랜잭션

#COMMIT

#ROLLBACK

#TCL

2. MySQL 오토커밋과 세션 가시성

MySQL은 기본적으로 오토커밋이 활성화되어 DML이 즉시 확정될 수 있으므로, 트랜잭션 제어가 필요하면 `autocommit`을 비활성화하고 명시적으로 `COMMIT/ROLLBACK`을 수행해야 합니다.

DBMS마다 트랜잭션 처리 기본값이 다를 수 있습니다. 강의에서는 Oracle과 MySQL을 비교하며, MySQL은 기본적으로 `autocommit=ON`인 경우가 많아 DML이 “쓰는 즉시 반영”되는 동작을 유발할 수 있음을 설명했습니다. 트랜잭션으로 묶어 처리하려면 오토커밋을 비활성화해야 합니다.

오토커밋 여부는 시스템 변수 조회로 확인합니다. 강의에서 제시된 흐름은 “조회 → OFF 설정 → 재조회”였습니다.

```
SHOW VARIABLES LIKE 'autocommit';
SET autocommit = FALSE;
SHOW VARIABLES LIKE 'autocommit';
```

또 하나의 핵심은 커밋 전 변경 사항의 가시성(**visibility**)입니다. 커밋하지 않은 DML 결과는 “DB에 확정 반영된 값”이 아니라 세션의 변경 상태에 가깝기 때문에, 같은 세션에서는 조회되지만 다른 세션에서는 조회되지 않을 수 있습니다.

이를 검증하기 위해 GUI(워크벤치 등)에서 INSERT 후 SELECT로 보이는 데이터를, 별도 접속(CMD의 `mysql` 클라이언트)에서 조회했을 때 보이지 않는 상황을 실습으로 확인했습니다. 이후 `COMMIT`을 수행하면 다른 세션에서도 동일 데이터가 조회됩니다.

CLI로 MySQL에 접속하는 과정도 함께 다루었습니다. 핵심은 MySQL 설치 경로의 `bin` 디렉터리에서 `mysql` 실행 파일로 접속한다는 점입니다.

```
mysql -u root -p
```

접속 후에는 DB 선택과 조회를 일반 SQL로 수행합니다.

```
USE testdb;
SELECT * FROM dpt;
```

이 파트에서 강조된 결론은 다음과 같습니다.

- 커밋 전 DML은 실행자 세션에서는 보일 수 있습니다.
- 커밋 전 DML은 다른 세션에서는 보이지 않을 수 있습니다.
- 커밋 이후에만 변경 사항이 “확정 데이터”로서 전체에 공유됩니다.

핵심 키워드

#autocommit

#세션

#가시성

#mysql CLI

#USE

3. INSERT 문법과 단일, 다중행 생성

INSERT는 단일행 생성뿐 아니라 다중행 생성도 가능하며, 컬럼명 명시 여부에 따라 값 매핑 방식과 생략 가능성이 달라지고 제약조건을 고려해야 합니다.

INSERT의 목적은 새로운 레코드를 생성하는 것입니다. 강의에서는 단일행 INSERT를 먼저 다룬 뒤, 다중행 INSERT를 두 가지 방식(서브쿼리, VALUES 리스트)으로 확장했습니다.

단일행 INSERT는 크게 두 가지 문법으로 구분됩니다.

구분	문법 형태	주요 특징
컬럼명 명시	<code>INSERT INTO t (c1,c2) VALUES (v1,v2);</code>	필요한 컬럼만 선택 입력이 가능하며, 미지정 컬럼은 NULL이 들어갈 수 있습니다.
컬럼명 생략	<code>INSERT INTO t VALUES (v1,v2,...);</code>	테이블에 정의된 컬럼 "순서대로" 1:1 매핑되며, 기본적으로 모든 컬럼 값을 제공해야 합니다.

컬럼명 명시 방식은 가독성이 높고 실수를 줄이기 쉬워 실무적으로 권장되는 방식으로 설명되었습니다. 다만 미지정 컬럼이 NULL로 들어가기 때문에, 해당 컬럼에 **NOT NULL 제약조건**이 있으면 생략할 수 없다는 점을 함께 정리해야 합니다.

컬럼명 생략 방식은 테이블 컬럼 순서를 정확히 알아야 하며, 컬럼 수만큼 값을 제공해야 오류가 나지 않습니다. 다만 "값을 주기 어려운 컬럼"이 있다면 명시적으로 `NULL` 값을 넣어 자리를 맞출 수 있다는 질의응답도 포함되었습니다.

또한 리터럴 표기 규칙도 실습 중 강조되었습니다. 숫자는 그대로 쓰지만, 문자열/날짜 등은 따옴표 처리가 필요합니다.

- 숫자 리터럴은 따옴표 없이 작성됩니다.
- 문자 및 날짜 계열 리터럴은 따옴표로 감싸야 합니다.

DPT 테이블 실습에서는 컬럼명 명시 INSERT로 데이터를 추가한 뒤, 커밋 전/후 가시성 차이를 다른 세션에서 확인했습니다. 또한 컬럼 일부를 생략했을 때 생략 컬럼이 NULL로 들어가는 동작을 확인했습니다.

다중행 INSERT는 두 가지로 확장됩니다.

방식	개념	핵심 포인트
서브쿼리 기반	<code>VALUES</code> 대신 <code>SELECT</code> 결과를 삽입합니다.	INSERT에서 사용하는 서브쿼리는 별도 괄호 없이 <code>SELECT ... FROM ...</code> 형태로 사용합니다.
VALUES 리스트	<code>VALUES (...), (...)</code> 처럼 여러 행을 나열합니다.	실행 결과로 영향을 받은 행(row) 수가 출력되며, 이는 JDBC에서 중요하게 활용됩니다.

서브쿼리 기반 다중행 INSERT 실습을 위해, 기존 테이블을 이용해 새 테이블을 만드는 CTAS(Create Table As Select)도 함께 소개되었습니다.

핵심 키워드

#INSERT

#단일행

#다중행

#CTAS

#제약조건

4. CTAS로 테이블 생성(구조, 데이터 복사)

CTAS는 기존 테이블의 SELECT 결과를 기반으로 새 테이블을 생성하며, 조건에 따라 레코드까지 복사하거나 구조만 생성할 수 있습니다.

강의에서는 다중행 INSERT 실습을 위한 준비로, 기존 테이블(EMP)을 이용해 새 테이블을 만드는 방법을 소개했습니다. 이 방식은 일반적으로 CTAS라고 부르며, “DDL을 본격적으로 배우기 전”에 실습 목적상 선행 소개되었습니다.

CTAS의 핵심은 다음과 같습니다.

- SELECT로 선택한 컬럼 구성대로 새 테이블의 컬럼이 정의됩니다.
- WHERE 조건이 참이면 레코드까지 포함해 생성할 수 있습니다.
- WHERE 조건이 거짓이면 레코드는 제외되고 구조만 생성됩니다.

레코드를 제외하고 “껍데기만” 만들기 위해 강의에서 사용한 전형적인 패턴은 `WHERE 1=2` 입니다. 이는 항상 거짓이므로 결과 레코드가 0건이 되어 구조만 생성됩니다.

```
CREATE TABLE my_emp AS
SELECT ename, sal, hiredate
FROM emp
WHERE 1 = 2;
```

이후 생성된 빈 테이블에 대해, 서버쿼리 기반 INSERT로 EMP의 데이터를 한 번에 적재하여 다중행 삽입을 확인했습니다. 이때도 DML이므로 커밋 전에는 다른 세션에서 보이지 않으며, 커밋 후에 공유된다는 트랜잭션 규칙을 동일하게 적용했습니다.

핵심 키워드

#CTAS

#CREATE TABLE AS SELECT

#WHERE 1=2

#구조복사

#데이터 복사

5. UPDATE 문법과 롤백 실습

UPDATE는 SET으로 컬럼 값을 변경하며 WHERE 유무에 따라 전체 행 또는 일부 행이 수정되고, 커밋 전에는 롤백으로 원복할 수 있습니다.

UPDATE는 “레코드 수정”이지만 정확히는 **컬럼 단위 수정**입니다. 강의에서는 WHERE절이 없는 업데이트와 있는 업데이트를 구분해 동작 차이를 강조했습니다.

UPDATE의 문법 골격은 다음과 같습니다.

```
UPDATE 테이블명
SET 컬럼1 = 값1, 컬럼2 = 값2
WHERE 조건;
```

WHERE절이 없으면 전체 레코드가 변경됩니다. 실습에서는 `my_emp` 테이블에 대해 이름과 급여를 전체 변경하는 UPDATE를 수행했고, 영향 받은 행 수가 출력되는 것을 확인했습니다. 또한 변경 후 커밋하지 않고 ROLLBACK을 수행하여 원복되는 동작을 확인했습니다.

WHERE절이 있으면 조건에 맞는 레코드만 변경됩니다. 예시로 사번이 특정 값(7369)인 행만 수정하는 실습을 수행했고, 마찬가지로 커밋 전에는 다른 세션에서 변경이 보이지 않으며 ROLLBACK으로 취소 가능함을 확인했습니다.

실습 중 자주 발생하는 문법 실수로, SET 절에서 컬럼 간 **쉼표 누락**이 대표 오류로 언급되었습니다.

- SET 절의 여러 컬럼 지정은 쉼표로 구분되어야 합니다.
- WHERE절 누락은 의도치 않은 전체 수정으로 이어질 수 있습니다.

핵심 키워드

#UPDATE

#SET

#WHERE

#전체수정

#부분수정

6. DELETE 문법과 영향 행 수

DELETE는 FROM 절로 대상 테이블을 지정하고 WHERE 유무에 따라 전체 삭제 또는 조건 삭제가 되며, 실행 시 영향 받은 행 수가 반환됩니다.

DELETE는 레코드를 삭제하는 DML입니다. UPDATE와 동일하게 WHERE절 유무가 매우 중요합니다.

DELETE의 기본 문법은 다음과 같습니다.

```
DELETE FROM 테이블명  
WHERE 조건;
```

WHERE절이 없으면 모든 레코드가 삭제됩니다. 실습에서는 `my_emp` 에서 전체 삭제를 수행한 뒤 SELECT로 빈 결과를 확인하고, 커밋하지 않은 상태에서 ROLLBACK으로 원복하는 흐름을 수행했습니다.

WHERE절이 있으면 조건에 맞는 레코드만 삭제됩니다. 예시로 특정 사번(7369, SMITH) 행만 삭제했으며, 이때 "1 row affected"처럼 **영향 받은 행 수**가 출력되는 점을 재차 강조했습니다. 이 값은 이후 JDBC에서 업데이트/삭제 결과 처리에 활용되는 중요한 정보입니다.

- INSERT/UPDATE/DELETE는 실행 결과로 영향 받은 레코드 수를 제공합니다.
- 커밋 전에는 변경이 타 세션에 반영되지 않습니다.
- 필요 시 롤백으로 작업을 취소할 수 있습니다.

핵심 키워드

#DELETE

#row affected

#WHERE

#전체삭제

#조건삭제

7. DDL 개요와 자동 커밋 특성

DDL은 테이블 등 DB 객체를 생성, 변경, 삭제하는 언어이며, 실행 시 자동 커밋되어 롤백이 불가능합니다.

DDL은 Data Definition Language로, 데이터 자체를 조작하기보다는 DBMS가 관리하는 **객체(Object)**를 정의/변경/삭제하는 목적을 가집니다. 강의에서는 DB 객체의 예로 테이블, 인덱스, 사용자 등을 언급했으며, 수업에서는 주로 테이블 생성/삭제 중심으로 다룰 것임을 밝혔습니다.

DML과 DDL의 용어 대응도 함께 정리되었습니다.

분류	영문	한국어	핵심 대상
DML	Data Manipulation Language	데이터 조작어	데이터(행/값)
DDL	Data Definition Language	데이터 정의어	객체(테이블 등)

DDL의 가장 중요한 특징은 **자동 커밋(auto-commit)**입니다. 즉 DDL은 실행 즉시 확정되며, ROLLBACK으로 되돌릴 수 없습니다. 따라서 트랜잭션으로 되돌릴 계획이 있는 DML 작업 중간에 DDL을 실행하면, 의도와 달리 확정이 발생할 수 있어 주의가 필요합니다.

핵심 키워드 **#DDL** **#객체** **#테이블** **#자동커밋** **#롤백불가**